[Project Title]

Nguyen Hoa Khang
524H0098
Department, Institution
524H0098@student.tdtu.edu.vn

Ly Gia Luan
524H0101
Department, Institution
524H0101@student.tdtu.edu.vn

Nguyen Thai Khoa
524H0109
Department, Institution
524H0109@student.tdtu.edu.vn

MSc. Nguyen Thanh An
Faculty of Information Technology, Ton Duc Thang University
nguyenthanhan@tdtu.edu.vn

***Abstract*—[State the project requirements and the three problems. Summarize the approaches used, the measured experimental results, and the completion level of each task. Use verified results and include no citations in the abstract.]**

***Index Terms*—[N-Queens], [adversarial search], [Kakuro], [constraint satisfaction]**

I. Introduction

[Introduce the project objectives, the three tasks, and the scope of this report. Briefly explain how the experimental results are evaluated.]

II. N-Queens with SAT and Local Search

A. Problem and Approach

The N-Queens problem requires placing N queens on an $N \times N$ chessboard so that no two queens attack each other. We implemented two approaches: a propositional-logic model solved using Glucose3 in PySAT [1], and Min-Conflicts local search [2]. We used the same solution verifier for both approaches.

1) Propositional variables and CNF constraints: Each square (r, c) is represented by a Boolean variable $Q_{r,c}$: true means it contains a queen. The N^2 primary variables are mapped to PySAT integer identifiers by

$$\mathrm{ID}(r, c) = rN + c + 1, \quad 0 \le r, c < N. \tag{1}$$

We require at least one queen in each row, $\bigvee_{c=0}^{N-1} Q_{r,c}$, and at most one queen in every row, column, and diagonal. For two distinct squares i, j belonging to the same group, the Pairwise at-most-one clause is $(\neg Q_i \vee \neg Q_j)$. The combination of exactly one queen per row and at most one per column gives exactly N queens in distinct columns.

To discover the conflicting squares, the implementation visits every board cell once and adds it to four groups: row r, column c, main diagonal $r - c + N - 1$, and anti-diagonal $r+c$. It then generates the row at-least-one clauses and encodes each at-most-one group with PySAT's `CardEnc.atmost`. We implemented three encodings: Pairwise (binary clauses with no auxiliary variables), Sequential Counter, and Ladder (both using auxiliary variables). For Pairwise encoding without symmetry breaking, counting the row, column, and diagonal pairs gives

$$C(N) = N + N^2(N-1) + \frac{N(N-1)(2N-1)}{3}. \tag{2}$$

Thus, its CNF clause count grows as $O(N^3)$. The optional symmetry-breaking clause restricts the first-row queen to the left half of the board, including the center when N is odd. This keeps a solution if one exists, but does not preserve all solutions for enumeration.

2) Glucose3 solving: After building the CNF, Glucose3 checks whether it is satisfiable. For a satisfying model, positive literals corresponding to board squares are decoded into queen positions, which are then validated using `verify_solution()`. We measure CNF construction, solver initialization, and solving separately, because the smallest CNF does not necessarily give the shortest runtime.

3) Min-Conflicts and the proposed heuristic: In local search, the state $S = [r_0, \ldots, r_{N-1}]$ stores the row r_c of the queen in column c. Therefore, one queen is always maintained in each column. We maintain row counts R and diagonal counts D_1, D_2 and compute the current conflicts of a queen as

$$f(r, c) = R[r] + D_1[r - c + N - 1] + D_2[r + c] - 3. \tag{3}$$

The subtraction removes the queen's own contribution. After choosing a queen, both strategies move it within its column to a row with the smallest resulting conflict count (randomly breaking ties). Standard Min-Conflicts randomly chooses any conflicted queen. Our *Most-Conflicted* heuristic instead chooses randomly among the queens with the highest conflict count: $c^* \in \arg\max_{c: f(r_c, c) > 0} f(r_c, c)$. We wanted to test whether fixing the most-conflicted queen first would reduce the search effort; this is a heuristic, not a guaranteed improvement. The code allows $\max(500, 30N)$ steps per attempt and up to 15 random restarts. Failure within this budget does not prove unsatisfiability.

B. Experimental Results and Discussion

We ran the benchmark in the application's Benchmark tab with **five trials per configuration**, using $N = 4, 8, 12, 16$ for both algorithm families. The Pairwise clause counts match

the encoding without symmetry breaking. The mean solving time measures only Glucose3's search, whereas the mean total time includes CNF construction and initialization. All 12 SAT configurations found a solution in 5/5 trials. Table I reports the displayed means and CNF sizes.

TABLE I
GLUCOSE3 RESULTS (5 TRIALS; TIMES IN MS)

N	Encoding	Vars	Clauses	Solve	Total
4	Pairwise	16	80	0.006	0.144
4	SeqCounter	54	108	0.004	0.124
4	Ladder	82	216	0.008	0.193
8	Pairwise	64	736	0.047	0.695
8	SeqCounter	270	592	0.059	0.594
8	Ladder	346	988	0.031	0.638
12	Pairwise	144	2608	0.064	1.347
12	SeqCounter	646	1460	1.263	2.272
12	Ladder	770	2272	1.109	2.495
16	Pairwise	256	6336	0.964	5.358
16	SeqCounter	1182	2712	4.562	6.292
16	Ladder	1354	4068	3.376	5.449

For $N = 16$, Sequential Counter uses 2712 clauses versus Pairwise's 6336, but its mean solve time is 4.562 ms versus 0.964 ms. This shows that a smaller clause count does not necessarily mean faster SAT solving. Conversely, for $N = 8$, Sequential Counter has a lower mean total time than Pairwise (0.594 versus 0.695 ms). The best choice depends on the metric and test size.

The local-search benchmark uses the same four board sizes and five trials for each strategy. Table II shows the means displayed in the application; all eight configurations were successful in 5/5 runs.

TABLE II
MIN-CONFLICTS RESULTS (5 TRIALS; TIMES IN MS)

N	Strategy	Success	Mean total
4	Standard	5/5	0.029
4	Most-Conflicted	5/5	0.032
8	Standard	5/5	0.328
8	Most-Conflicted	5/5	0.183
12	Standard	5/5	0.184
12	Most-Conflicted	5/5	0.221
16	Standard	5/5	0.445
16	Most-Conflicted	5/5	0.235

Most-Conflicted had a lower mean time at $N = 8$ and $N = 16$, but Standard was faster at $N = 4$ and $N = 12$. Therefore, the five-trial results do not establish that our heuristic is consistently faster. These short runtimes can also be affected by measurement noise; additional trials and larger boards would make the comparison more reliable. The application also supports CSV export, which we can use for further analysis of steps and median runtimes. Overall, the two methods produced valid placements, while their performance depended on encoding or variable-selection choices.

III. ADVERSARIAL SEARCH FOR TANK BATTLE

A. Problem and Approach

[Describe the game state, available actions, evaluation, and implemented search methods, including Minimax and Alpha-Beta where applicable. Explain assumptions and limitations relevant to the implementation.]

B. Experimental Results

[State the scenarios, configurations, measurement method, and observed results. Use comparable settings and identify any approximations.]

TABLE III
TANK BATTLE EXPERIMENTAL RESULTS

Method	[Metric 1]	[Metric 2]
[Method]	[Value]	[Value]

IV. KAKURO AS A CONSTRAINT SATISFACTION PROBLEM

A. Problem and Approach

[Define variables, domains, and constraints. Explain Backtracking and the roles of Forward Checking, MRV, Degree, and LCV as implemented. Cite sources.]

B. Experimental Results

[Identify the board, verification method, measured results, and limitations. Do not claim that the implementation uses AC-3 or proves uniqueness unless those claims are supported by the implementation and experiment.]

TABLE IV
KAKURO EXPERIMENTAL RESULTS

Board	[Metric]	Result
[Board]	[Value]	[Result]

V. CONTRIBUTIONS

[Describe each member's individual work and completion level. Add one row per member; percentages must reflect the group's actual assessment.]

TABLE V
INDIVIDUAL CONTRIBUTIONS

Member / ID	Individual work	Completion
[Name / ID]	[Tasks and responsibilities]	[0–100%]
[Name / ID]	[Tasks and responsibilities]	[0–100%]
[Name / ID]	[Tasks and responsibilities]	[0–100%]

VI. SELF-EVALUATION

[Evaluate each task against the assignment requirements. Record completion, missing work, and the estimated score using the assignment's actual rubric.]

TABLE VI
TASK COMPLETION AND ESTIMATED SCORES

Task	Completion	Score	Maximum
Task 1	[%]	[Score]	[Max]
Task 2	[%]	[Score]	[Max]
Task 3	[%]	[Score]	[Max]
Report / interview	[%]	[Score]	[Max]

VII. Conclusion

[Summarize the project requirements, approaches, measured results, and completion levels. State important limitations and possible future work.]

References

[1] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," in *Theory and Applications of Satisfiability Testing – SAT 2018*, 2018, pp. 428–437.

[2] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, "Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems," *Artificial Intelligence*, vol. 58, no. 1–3, pp. 161–205, 1992.